Micro:bit Hex Flashing Reference Guide

Smart Car Firmware -- Complete Mapping for All 16 Control Modes

ProjectRobot1_2026 | Micro:bit + K210 Smart Car

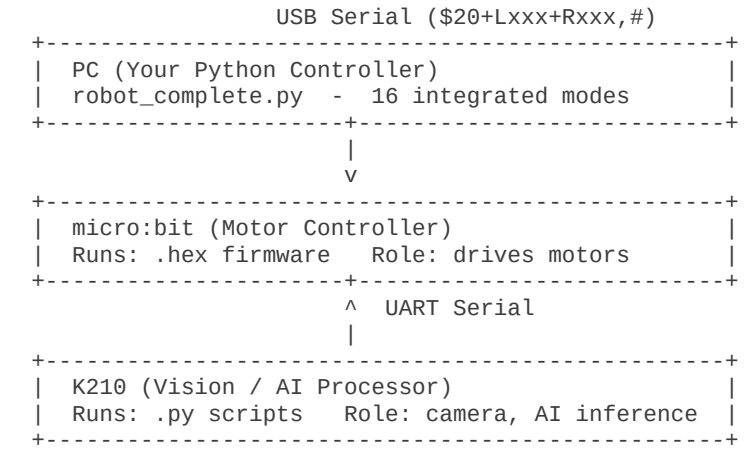
Version: 1.0 Date: June 2026

Hardware: micro:bit V2 | K210 (Sipeed Maix) | Tiny:bit Car Chassis

Repository: ProjectRobot1_2026

1 System Architecture

The robot system consists of three computing nodes that communicate over serial (UART):



Key insight: Both the PC and the K210 send motor commands to the micro:bit using the $20+Lxxx+Rxxx,# protocol. ALL car-experiment .hex files interpret this format. The difference between .hex files is which additional vision-command codes they handle (e.g. $01 for colour, $04 for AprilTag).

2 Complete Mode-to-Hex Mapping

2.1 PC-Only Modes (No K210 Required)

These modes send motor commands directly from your PC to the micro:bit via USB serial. Flash ANY ONE of the car-experiment .hex files -- they all handle $20 motor commands.

#	Mode	PC Program	micro:bit Hex	K210 Script	Type
1	Basic Movement	robot_basic_move.py	k210_color.hex	--	PC
2	Direction Control	robot_direction_control.py	(any .hex works)	--	PC
3	Speed Control	robot_speed_control.py		--	PC
4	Path Control	robot_path_control.py		--	PC
14	Voice Command	robot_voice_command.py		--	PC

2.2 Vision Modes (K210 Required)

These modes require the K210 camera to run a matching Python script. The K210 processes images and sends results to the micro:bit over UART.

#	Mode	PC Program	micro:bit Hex	K210 Script	KMODEL	Type
5	Colour Tracking	robot_color_tracking.py	k210_follow_color.hex	3.11_follow_color.py	--	K210
6	Line Following	robot_line_following.py	k210_color_line.hex	3.9_color_follow_line.py	--	K210
7	AprilTag Nav	robot_apriltag_navigation.py	k210_follow_apriltag.hex	3.10_follow_apriltag.py	--	K210
8	QR Code Comman	robot_qrcode_commander.py	k210_qrcode_oder.hex	2.2_3.3_find_qrcodes.py	--	K210
9	Face Recognition	robot_face_recog_control.py	k210_Face_detection.hex	2.8_face_recog.py	3 kmodels*	K210
10	Object Detection	robot_object_detect_nav.py	k210_object_detect.hex	2.5_3.4_voc20_object_dete	1 kmodel	K210
11	Gesture Control	robot_gesture_control.py	k210_Face_detection.hex	3.7_yolo_face_detect-Y.py	1 kmodel	K210
12	MNIST Digit Contrc	robot_mnist_control.py	k210_Handwritten_digital_co	2.9_3.8_mnist.py	1 kmodel	K210
16	Self-Learning Nav	robot_self_learning_nav.py	k210_color.hex	2.6_3.5_self_learning.py	1 kmodel	K210

** Face recognition needs 3 kmodels: face_detect_320x240.kmodel, ld5.kmodel, feature_extraction.kmodel*

2.3 Simulation-Only Modes (No Hardware Required)

These modes run entirely on the PC with simulated sensor data. No hex flashing or K210 needed.

#	Mode	PC Program	Description	Type
13	Obstacle Avoidance	robot_obstacle_avoidance.py	Virtual obstacles with 4 avoidance strategies	SIM
15	Autonomous Explore	robot_autonomous_explore.py	Simulated occupancy grid & exploration	SIM

3 Additional K210 Features

Beyond the 16 integrated modes, the source code includes these standalone K210 functions:

Function	micro:bit Hex	K210 Script	KMODEL
Colour Recognition (RGB)	k210_color.hex	3.1_color_rgb.py or 2.1_color_recognition.py	--
Barcode Scanning	k210_barcode.hex	2.2_3.2_find_barcodes.py	--
Face Mask Detection	k210_Mask_detection.hex	2.7_3.6_face_mask_detect.py	detect_5.kmodel
Road Sign (AI Sport)	k210_Road_sign_indicating_action.hex	3.12_tinybit_AI_sport.py	tinybit_AI_0x.kmodel
AI Line Following	k210_tinybit_AI_line.hex	--	tinybit_AI_0x.kmodel

4 Quick-Start Guide

4.1 Minimal Setup -- PC Control Only (Modes 1-4, 14)

1. Flash the micro:bit -- Drag k210_color.hex (or any car hex) onto the micro:bit drive.
2. Connect the PC -- Plug the micro:bit into your PC via USB cable.
3. Run -- Execute "python robot_complete.py" and select modes 1, 2, 3, 4, or 14.
4. No K210 needed for these modes.

4.2 Vision Setup -- Colour Tracking Example (Mode 5)

1. Flash the micro:bit -- k210_follow_color.hex
2. Prepare K210 SD card -- Copy 3.11_follow_color.py to SD root. Rename to main.py for auto-boot.
3. Wire the hardware -- Connect K210 UART TX/RX to micro:bit serial pins.
4. Power on -- Both K210 and micro:bit.
5. Run -- "python robot_complete.py" and select mode 5.

4.3 AI Vision Setup -- Object Detection Example (Mode 10)

1. Flash the micro:bit -- k210_object_detect.hex
2. Prepare K210 SD card -- Copy 2.5_3.4_voc20_object_detect.py to root as main.py.
3. Copy model files -- Create folder /sd/KPU/voc20_object_detect/ on SD card, place voc20_detect.kmodel inside.
4. Wire & power -- Connect K210 to micro:bit, power both on.
5. Run -- "python robot_complete.py" and select mode 10.

Important: The K210 script path inside the .py file must match the SD card folder structure. For example, kpu.load_kmodel("/sd/KPU/voc20_object_detect/voc20_detect.kmodel") means the file must be at X:\KPU\voc20_object_detect\voc20_detect.kmodel on the SD card.

5 micro:bit Hex File Quick Reference

Hex File	Protocol Codes Handled	Best For Modes
k210_color.hex	$20 (motor)	1, 2, 3, 4, 14, 16
k210_follow_color.hex	$20, $01 (colour)	5
k210_color_line.hex	$20	6
k210_follow_apriltag.hex	$20, $04 (AprilTag)	7
k210_qrcode_oder.hex	$20, $03 (QR)	8
k210_barcode.hex	$20, $02 (barcode)	8 (barcode variant)
k210_Face_detection.hex	$20, $08, $14 (face)	9, 11
k210_object_detect.hex	$20, $09 (object)	10
k210_Handwritten_digital_control.hex	$20, $11 (MNIST)	12
k210_Mask_detection.hex	$20, $07 (mask)	Mask detection
k210_Road_sign_indicating_action.hex	$20, $09 (sign)	Road sign / AI sport
k210_tinybit_AI_line.hex	$20	AI line follow

6 KMODEL Files Directory Structure

Copy the following folders from source code/KPU/ to the K210 SD card, preserving the exact paths:

```
SD Card Root
+-- main.py                      (K210 script, renamed from the .py you need)
+-- KPU/
|   +-- voc20_object_detect/
|   |   +-- voc20_detect.kmodel
|   +-- yolo_face_detect/
|   |   +-- face_detect_320x240.kmodel
|   |   +-- yolo_face_detect.kmodel
|   +-- face_recognization/
|   |   +-- ld5.kmodel
|   |   +-- feature_extraction.kmodel
|   +-- face_mask_detect/
|   |   +-- detect_5.kmodel
|   +-- mnist/
|   |   +-- uint8_mnist_cnn_model.kmodel
|   +-- self_learn_classifier/
|       +-- mb-0.25.kmodel
+-- tinybit_AI_01.kmodel
+-- tinybit_AI_02.kmodel
+-- tinybit_AI_03.kmodel
+-- tinybit_AI_04.kmodel
```

Camera note: Use tinybit_AI_04.kmodel for the GC2145 camera (newer version). Use tinybit_AI_01/02/03.kmodel for the OV2640 camera (older version).

7 Troubleshooting

Symptom	Likely Cause	Solution
Robot does not move	Wrong serial port or baud rate	Check COM port in Device Manager; ensure 115200 bps
Robot moves but vision modes do not	K210 script not running or mismatched .hex	Verify K210 script is running; match .hex to mode per Section 2.2
K210 reports "model not found"	KMODEL path mismatch on SD card	Check load_kmodel() path in K210 .py; ensure .kmodel at exact path
Motors spin in wrong direction	Motor polarity reversed	Swap motor wires or negate speed values in code
Serial "access denied" error	Another program has COM port open	Close MaixPy IDE, Mu Editor, or any terminal using the port

AI detection very slow (< 5 FPS)	KMODEL too large or insufficient memory	Use smaller model variant; reduce image resolution